

Backtracking

- Backtracking is an algorithmic technique that tries all possible solutions to find one or all valid answers.
- It uses recursion to explore different choices step by step.
- At each step, make one choice and continue solving the remaining problem.
- If the current choice leads to a dead end or invalid solution, backtrack by undoing the last choice.
- After undoing, try the next available choice until all possibilities are explored.
- Every backtracking problem follows the pattern: Choose → Explore → Undo → Choose Next.
- Backtracking is suitable for problems involving permutations, combinations, subsets, paths, and constraint satisfaction.
- It is commonly used in problems like N-Queens, Sudoku Solver, Rat in a Maze, Permutations, Subsets, and Combination Sum.
- Backtracking often explores a decision tree (state-space tree) where each root-to-leaf path represents a possible solution.
- Backtracking algorithms usually have exponential time complexity (e.g., $O(2^n)$ or $O(n!)$) because they may need to explore many possible choices.

Rat in a Maze (Backtracking)

- Problem Statement
 - A rat is placed at the top-left corner (0,0) of an $N \times N$ maze. The rat has to reach the **bottom-right corner (N-1,N-1)**.
 - 1 → Open cell (can move)
 - 0 → Blocked cell (cannot move)
 - The rat can move in four directions:
 - Up (U)
 - Down (D)
 - Left (L)
 - Right (R)
 - Find all possible paths from source to destination.

Algorithm

- Start from (0,0).
- If destination is reached, store the path.
- Mark the current cell as visited.

- Try all four directions:
 - Down
 - Left
 - Right
 - Up
- If the move is valid, recursively continue.
- After recursion, unmark the cell (Backtracking).

Java Solution

```
import java.util.ArrayList;

public class RatInMaze {

    static ArrayList<String> ans = new ArrayList<>();

    static void solve(int[][] maze, int row, int col,
                     boolean[][] visited, String path) {

        int n = maze.length;

        // Destination reached
        if (row == n - 1 && col == n - 1) {
            ans.add(path);
            return;
        }

        visited[row][col] = true;

        // Down
        if (isSafe(maze, row + 1, col, visited))
            solve(maze, row + 1, col, visited, path + "D");

        // Left
```

```

    if (isSafe(maze, row, col - 1, visited))
        solve(maze, row, col - 1, visited, path + "L");

    // Right
    if (isSafe(maze, row, col + 1, visited))
        solve(maze, row, col + 1, visited, path + "R");

    // Up
    if (isSafe(maze, row - 1, col, visited))
        solve(maze, row - 1, col, visited, path + "U");

    // Backtracking
    visited[row][col] = false;
}

static boolean isSafe(int[][] maze, int row, int col,
    boolean[][] visited) {

    int n = maze.length;

    return row >= 0 &&
        col >= 0 &&
        row < n &&
        col < n &&
        maze[row][col] == 1 &&
        !visited[row][col];
}

public static void main(String[] args) {

    int[][] maze = {
        {1,0,0,0},
        {1,1,0,1},
        {1,1,0,0},
        {0,1,1,1}
    };

```

```
int n = maze.length;

if (maze[0][0] == 1) {
    boolean[][] visited = new boolean[n][n];
    solve(maze, 0, 0, visited, "");
}

System.out.println(ans);
}
```